CS4475 Fall 2026
Alexa Ramirez-Salinas
aramirezsa@miners.utep.edu

Lab Report
Submitted on Oct. 29, 2026

# PA3: Parallel MergeSort

**Task 1**. (10 pts) **Speedup analysis**

Q1. Run the code with a large enough input to get several seconds of run time for the sequential mergesort.

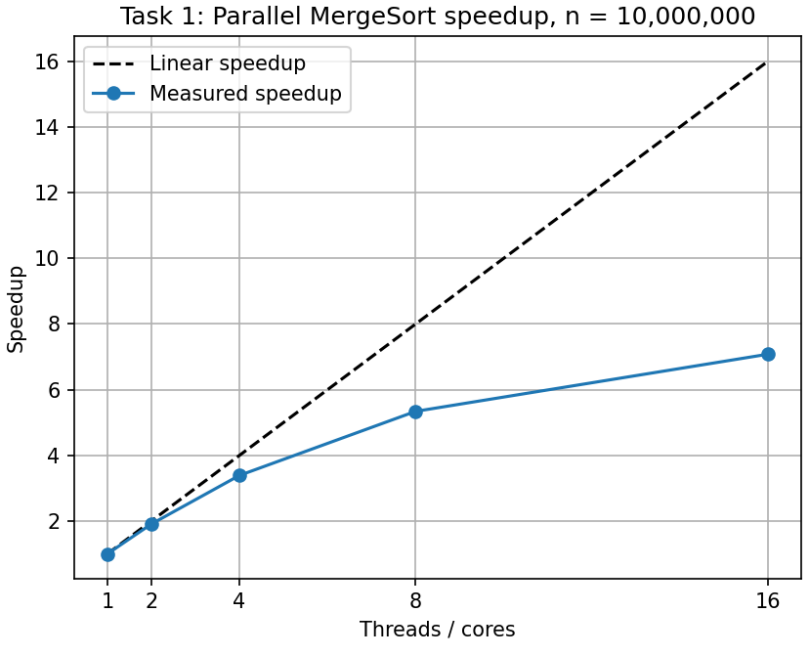
I used an input of 10,000,000 doubles. Sequential mergesort took about 1.53 s, which is shorter than the "several seconds" the assignment suggests. It was still long enough for stable timings, since run-to-run variation was only a few milliseconds.

Q2.Run with 1, 2, 4, 8, and 16 cores on a compute node. Do each run a few times and average the results.

I ran the code on a Bridges-2 compute node with 16 cores (via interact). I built five executables with THREAD_LEVEL 0 through 4 to get 1, 2, 4, 8, and 16 threads, and ran each five times. I averaged the times.

| Threads | Avg sequential (s) | Avg parallel (s) | Speedup | Linear | Efficiency |
|---|---|---|---|---|---|
| 1 | 1.5331 | 1.535 | 1 | 1 | 100% |
| 2 | 1.5326 | 0.8004 | 1.91 | 2 | 96% |
| 4 | 1.5325 | 0.4524 | 3.39 | 4 | 85% |
| 8 | 1.5336 | 0.2873 | 5.34 | 8 | 67% |
| 16 | 1.5332 | 0.2165 | 7.08 | 16 | 44% |

Q3. Calculate and graph the speedup and compare it to linear speedup



Speedup is the average sequential time divided by the average parallel time. The program tracks linear speedup closely at 2 threads (96% efficiency) and reasonably at 4 threads (85%). It then falls away from linear: efficiency is 67% on 8 threads and only 44% on 16 threads. Going from 8 to 16 threads cut the time by about 25% rather than 50%. The main reason is that every merge in the recursion tree is sequential, so the top-level merge processes all 10 million elements on one thread regardless of core count. Mergesort is also memory-bound, so threads likely compete for memory bandwidth, and the compute node is shared. An earlier set of runs gave about 5.6 at 16 threads and nearly identical results up to 8 threads, so 16-thread timings vary between sessions.

**Task 2**. (15 pts) **Amdahl's Law**

Q1. Put timing calls around the top-level call to the sequential merge and report the time.
I added gettimeofday calls around the top-level merge in the parallel code and printed the elapsed time. It was about 0.065 s at every thread count (0.0646, 0.0648, 0.0649, and 0.0648 s for 2, 4, 8, and 16 threads).

Q2. Estimate the proportion of time spent in the sequential merge for the 10,000,000 input.
Compared with the sequential mergesort time of about 1.53 s, the top-level merge accounts for about 4.2% of the sequential run time (0.0648 / 1.533).

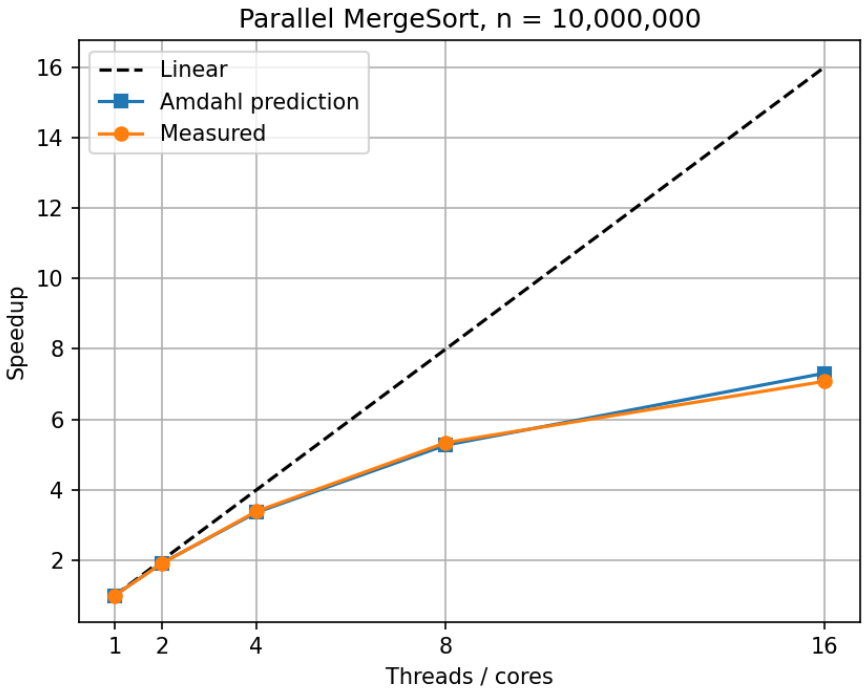
Q3. Why do you need additional calculation on the measured merge time to estimate the time spent in sequential merge in the parallel execution?
The 0.065 s cannot be used alone as the serial time of the parallel run. Each parent thread must wait for its two children before it merges, so the sequential merges at every level of the tree lie on the critical path, not only the top one. Those merges handle n, n/2, n/4, and so on elements, so their times are M, M/2, M/4, and so on. With $\log_2(p)$ levels they sum to $2M(1 - 1/p)$.

Q4. Use Amdahl's Law to predict the maximum speedup with infinitely many cores and with the number of cores you are using.

| p | Top-level merge M (s) | Serial time 2M(1-1/p) (s) | Serial fraction f | Amdahl S(p) | Measured S(p) |
|---|---|---|---|---|---|
| 1 | n/a | 0 | 0 | 1 | 1 |
| 2 | 0.0646 | 0.0646 | 0.0421 | 1.92 | 1.91 |
| 4 | 0.0648 | 0.0973 | 0.0635 | 3.36 | 3.39 |
| 8 | 0.0649 | 0.1136 | 0.0741 | 5.27 | 5.34 |
| 16 | 0.0648 | 0.1215 | 0.0792 | 7.31 | 7.08 |
| ∞ | 0.0648 | 0.1296 | 0.0845 | 11.8 (=1/f) | n/a |

Amdahl's Law is $S(p) = 1 / (f + (1 - f)/p)$. On 16 cores the predicted maximum speedup is 7.31. With infinitely many cores it is $1/f \approx 11.8$.

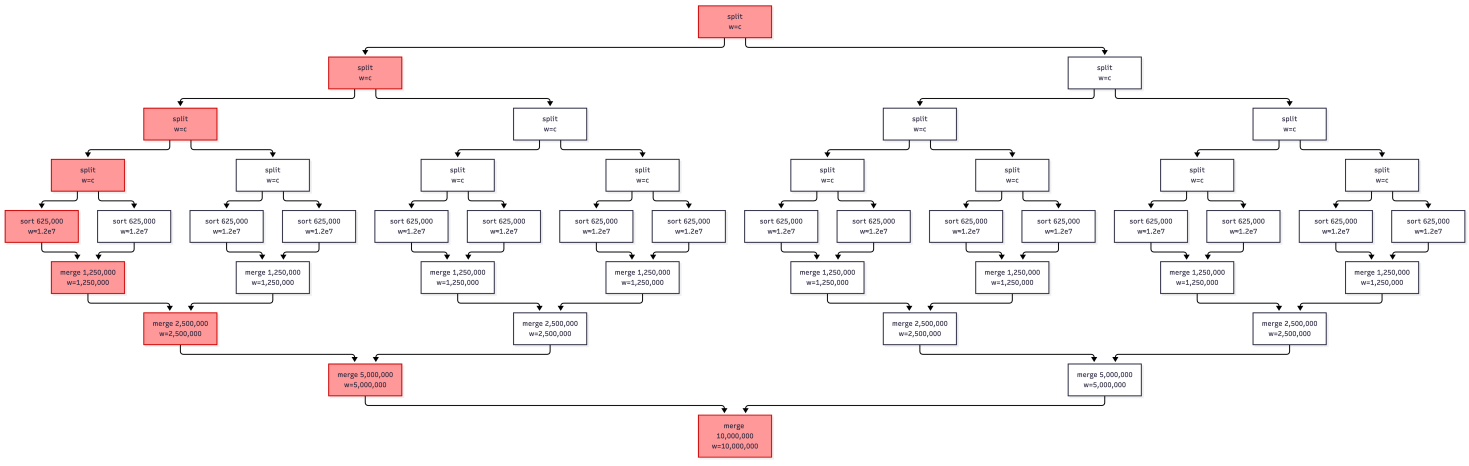
Q5. Graph and discuss how your Task 1 speedups compare with your Amdahl's Law predictions.




The measured speedups of 1.91, 3.39, 5.34, and 7.08 are within about 3% of the Amdahl predictions of 1.92, 3.36, 5.27, and 7.31. The measured value is slightly above the prediction at 4 and 8 threads, probably because the smaller merges lower in the tree fit better in cache than the top-level merge I used to estimate them. It is slightly below at 16 threads, likely because of thread creation and join overhead, memory-bandwidth contention, and the shared node. The leaf sorts also share the same memory system, so they are not perfectly parallel. Overall, the sequential merge explains most of the gap between my Task 1 speedups and linear speedup, and it caps this implementation near 11.8× no matter how many cores are used.

**Task 3.** (15 pts) **DAG and work-span analysis (p = 16)**

Q1. Draw a computational DAG that represents the parallel execution.



For 16 threads (THREAD_LEVEL 4) and n = 10,000,000, the DAG has three parts. At the top is a binary tree of 15 split nodes, each creating two child threads. Below it are 16 leaf nodes, each sequentially sorting n/16 = 625,000 elements. Below those is a binary tree of 15 merge nodes, where each merge depends on its two children.

Q2. Label the work for each node (nodes do not have constant work).

| Node type | Count | Size each | Work per node | Total work |
|---|---|---|---|---|
| Split (create 2 threads) | 15 | n/a | c (negligible) | 15c |
| Leaf sequential sort | 16 | 625,000 | $(n/16)\cdot\log_2(n/16) \approx 1.20\times10^7$ | $\approx 1.93\times10^8$ |
| Merge, level 1 | 8 | 1,250,000 | $1.25\times10^6$ | $1.0\times10^7$ |
| Merge, level 2 | 4 | 2,500,000 | $2.5\times10^6$ | $1.0\times10^7$ |
| Merge, level 3 | 2 | 5,000,000 | $5.0\times10^6$ | $1.0\times10^7$ |
| Merge, level 4 (top) | 1 | 10,000,000 | $1.0\times10^7$ | $1.0\times10^7$ |

A split costs a negligible constant c. A leaf sort of m elements costs $m\cdot\log_2(m)$. A merge of m elements costs m.

Q3. Carry out the work-span analysis: add up the work and find the critical path length.
The total work is $T_1 = 15c + 16(1.20\times10^7) + 4(10^7) \approx 2.33\times10^8$, which equals $n\cdot\log_2(n)$. This is the same as sequential mergesort, so the parallel version does no extra work.

The critical path goes through four splits down one branch, one leaf sort of $1.20\times10^7$, and then the merges of n/8, n/4, n/2, and n elements, which total $1.875\times10^7$. The span is $T\infty \approx 4c + 1.20\times10^7 + 1.875\times10^7 \approx 3.08\times10^7$.

| Quantity | Value |
|---|---|
| Work $T_1$ | $n\cdot\log_2(n) \approx 2.33\times10^8$ |
| Span $T\infty$ (4 splits + 1 leaf + merges n/8, n/4, n/2, n) | $\approx 1.20\times10^7 + 1.875\times10^7 \approx 3.08\times10^7$ |
| Parallelism $T_1/T\infty$ | $\approx 7.5$ |
| Measured speedup at 16 threads | 7.08 |

Q4. Discuss the critical path and its implications (lower bounds on parallel execution time).
The parallelism of about 7.5 means no schedule can achieve more than about a 7.5× speedup on this DAG. Brent's bound gives a guaranteed speedup of about 5.1 on 16 processors, so the achievable speedup lies between 5.1 and 7.5. My measured 7.08 falls in that range and is just below the upper bound.

Converting units to time at about 6.6 ns per unit (1.53 s divided by $2.33\times10^8$), the span predicts about 0.20 s against a measured parallel time of 0.217 s, and the top-level merge predicts about 0.066 s against a measured 0.065 s. More than half of the span is the chain of sequential merges, so that chain is the critical path, and this is why extra threads help so little. It agrees

with the Amdahl analysis in Task 2. To increase parallelism, the sequential merge would need to be replaced by a parallel merge, which would reduce the span from O(n) to polylogarithmic.

**Task 4**. (15 pts) **Reflection**

The hardest parts of this assignment were practical rather than conceptual. I had to learn the Bridges-2 workflow of logging in with ssh and then using interact to get a compute node. At first I ran programs on the shared login node, which gives unreliable timings, so I moved the runs to a compute node. I also pasted several commands while the interact job was still queued, and they got jumbled, so afterward I entered one command at a time. My first Task 2 edit put the timing code in the wrong place. It replaced the leaf-level sequential sort with a call to merge using an uninitialized midpoint. The build failed, and after a quick fix the programs crashed with free( ): invalid size and a segmentation fault. I found the bug by printing the function with `sed` and comparing it to the original. I then moved the timing code to the final merge at the bottom of the function and checked that every compile was error-free before running experiments. Copying files off the cluster failed with "subsystem request failed" and "scp: command not found" on the login node, and I solved it by using PSC's data transfer node, data.bridges2.psc.edu. I also ran a loop twice and appended to the same results file, and I had to keep each assignment on its own git branch. From this assignment I learned that speedup is limited by the sequential fraction of a program. The sequential merge caps parallel mergesort at about 7× on 16 cores and about 11.8× on infinitely many. My Amdahl prediction came within 3% of the measurement once I summed the merge levels as $2M(1 - 1/p)$ instead of using only the top-level merge. The work-span analysis gave the same conclusion from another angle: work of $n \cdot \log_2(n)$, span of about $3.1 \times 10^7$, and parallelism of about 7.5 explain why 16 cores cannot give a 16× speedup. I also learned that a better algorithm, a parallel merge, would help more than adding threads. Good measurement needs repeated runs, identical compile flags, a compute node, and checks for correctness before trusting the numbers.

I used an AI assistant (Claude) to guide me through this assignment. It walked me through the workflow step by step: setting up the pa3 branch, building the different thread-count executables, adding timing code, running the experiments on Bridges-2, and working out the Amdahl's Law and work-span calculations. It also helped me debug problems, including the compile error and memory crash from my misplaced timing edit, and the file-transfer failures. I ran all the experiments myself on Bridges-2 and the results come from my own measurements. I checked the calculations against my output files, and I used the AI's explanations to understand the concepts (serial fraction, the $2M(1 - 1/p)$ merge-chain sum, work, span, and parallelism) rather than only copying numbers. The final write-up and interpretation of the results are my own.